PROJECT SYNOPSIS

"AGILEGRAPH"

Graph-Learned Crypto-Agility Risk Scoring for Post-Quantum Migration

Students	1. Spoorthi — NNM23CB058 2. Sindhoora R — NNM23CB057 3. Nivedita V Shetty — NNM23CB033 4. Sujan S — NNM23CB062
Guide Name	
Department	Department of Computer Science (Cyber Security)
Institution	NMAM Institute of Technology (NMAMIT), Nitte (Deemed to be University), Karnataka
Academic Year	2023 – 27 (Final Year, B.Tech)

1. Introduction

The security of nearly every digital system in use today — banking transactions, healthcare records, government communications, and everyday web traffic — rests on public-key cryptographic algorithms such as RSA and ECC. Large-scale, fault-tolerant quantum computers are expected to be capable of breaking these algorithms using Shor's algorithm, and while such machines do not yet exist at the required scale, the risk is not purely theoretical. Adversaries can already record encrypted traffic today and decrypt it once sufficiently powerful quantum computers become available, a strategy widely referred to as "harvest-now, decrypt-later." In response, the U.S. National Institute of Standards and Technology (NIST) finalized its first set of post-quantum cryptography (PQC) standards in August 2024 — ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205) — giving organizations quantum-resistant algorithms to migrate to [1], [2], [3].

However, having quantum-safe algorithms available does not solve the migration problem by itself. Most organizations do not have a reliable inventory of where cryptography is actually used across their code, certificates, libraries, and network endpoints, which makes planning a migration extremely difficult [4], [5]. This project, AgileGraph, addresses that gap by combining automated cryptographic discovery with a graph-based AI model that scores and ranks cryptographic risk across an organization, producing a clear, explainable, and prioritized migration roadmap.

2. Problem Statement

Organizations urgently need to migrate to post-quantum cryptography, but most do not know where their vulnerable cryptography is located in the first place — it is scattered across old libraries, forgotten certificates, and code that has not been touched in years. Discovering this cryptography today is a largely manual process: security teams search codebases, check certificates, and inspect network traffic by hand, which is slow, error-prone, and does not scale. Even when weak cryptography is found, existing tools present it as a flat, unprioritized list — telling a team that "this certificate is outdated" but not that "fixing this certificate first will protect the most sensitive data with the least effort." There is currently no open, explainable system that models how cryptographic weaknesses propagate across an interconnected system of code,

certificates, libraries, and sensitive data, and uses that model to produce a defensible, risk-ranked migration order.

3. Objectives

- To design and implement an automated cryptographic discovery and risk assessment framework that scans source code, live network endpoints/certificates, and dependency manifests across Java, Python, and Go projects; constructs a heterogeneous cryptographic risk graph comprising six categories of entities and seven relationship types; and generates an open, reproducible heuristic risk score using measurable and auditable factors including data sensitivity, asset criticality, internet exposure, cryptographic algorithm strength, known vulnerabilities (CVEs), dependency centrality, and migration effort.
- To develop a Graph Attention Network (GATv2) that uses the heuristic risk score as node initialization and a weak supervisory signal to refine post-quantum migration priorities by propagating contextual risk across shared libraries, certificates, and dependency relationships, and to empirically verify — via a controlled ablation that trains the GNN with and without the heuristic score present as an input feature — that the model's gains come from graph structure rather than from reproducing its own training signal.
- To evaluate the proposed framework by benchmarking the GNN-refined prioritization against the heuristic baseline, simpler rule-based ranking approaches, and the ranking produced by an existing open cryptographic-inventory tool (CBOMkit), quantifying the impact of graph-based contextual propagation with bootstrap confidence intervals and a paired significance test, and validating the results using an independently collected, held-out expert-annotated dataset with categorical (High / Medium / Low) risk labels, reporting inter-annotator agreement among the experts themselves as a reliability ceiling for the model-agreement numbers.
- To develop an explainable web-based decision support system that presents prioritized migration recommendations, plain-language explanations generated from attention-weight/GNNExplainer attribution combined with a decomposition of the underlying heuristic factors, estimated risk reduction for each asset, and a separate Mosca Readiness Index — an organization-configurable planning indicator based on data confidentiality lifetime, migration

duration, and an adjustable assumed quantum-capability horizon — kept outside the GNN training loop so that the core methodology remains independent of speculative quantum timeline assumptions.

4. Literature Survey

4.1 Existing System

Cryptographic discovery today is largely handled through manual audits or commercial scanning/inventory tools such as Keyfactor and AppViewX, which catalogue certificates and keys but stop short of modelling how a weakness in one component affects the rest of the system. The emerging Cryptography Bill of Materials (CBOM) concept, standardized as part of CycloneDX 1.6 in 2024 (published as ECMA-424), formalizes cryptographic inventory in a machine-readable format and is supported by open tools such as CBOMkit [9]. NIST's own National Cybersecurity Center of Excellence (NCCoE) migration guidance similarly emphasizes cryptographic discovery and crypto-agility as first steps in PQC migration planning [4].

4.2 Limitations of Existing System

- Discovery tools identify cryptographic assets but produce flat, unprioritized lists with no understanding of downstream impact.
- Risk-scoring formulas in commercial tools are closed and vendor-specific, making them difficult to audit or trust.
- No existing tool models cross-system dependency — i.e., what breaks elsewhere if a given certificate, library, or key is not migrated.
- Explanations for why an item is flagged as risky are rarely provided, resulting in a “black box” experience for security teams.
- Migration-order simulation (comparing outcomes of different fix sequences) is largely absent from current tooling.
- Where risk formulas do exist, they are typically presented as ground truth rather than as an auditable heuristic, leaving no principled role for a learned model to add value beyond the formula itself.

- No open tool reports a direct, quantitative comparison against a learned, graph-propagated ranking — the gap this project targets empirically.

4.3 Related Research Papers

Hasan et al. propose a framework for migrating to post-quantum cryptography that performs security-dependency analysis and presents real-world case studies, motivating the need for dependency-aware migration planning [4]. Alnahawi et al. survey the current state of crypto-agility, characterizing how prepared organizations are to swap cryptographic primitives without large-scale rewrites [5]. Näther et al. conduct a systematic literature review of software migration approaches toward post-quantum cryptography, cataloguing existing tools and identifying open gaps in automated migration tooling [6]. On the AI side, Brody, Alon, and Yahav introduce GATv2, a dynamic graph-attention variant that is strictly more expressive than the original Graph Attention Network and forms the core learning architecture proposed in this project [7]. Motie and Raahemi systematically review the use of graph neural networks for financial fraud detection, illustrating how GNN-based risk propagation across heterogeneous graphs has proven effective in adjacent risk-scoring domains such as fraud and credit risk, including settings where model supervision comes from noisy or proxy labels rather than exact ground truth [8]. The OWASP CycloneDX project formalizes the Cryptography Bill of Materials as a machine-readable standard for cryptographic asset inventory, which this project extends with graph-based risk propagation rather than flat inventory alone [9]. Most directly related, recent work on quantum-safe code auditing combines regex-based detection, LLM-assisted enrichment, and quantum-aware risk scoring for PQC migration, evaluated across five open-source libraries [10]; that work is a concurrent, non-peer-reviewed preprint and is treated here as motivation for timeliness rather than as a validated benchmark.

5. Proposed System

AgileGraph is structured as four cooperating layers. The scanning and data-collection layer statically analyses source code to detect cryptographic primitives (RSA, ECDSA, AES, and others) across Java, Python, and Go projects, actively scans authorized live servers for certificate and TLS configuration weaknesses, and parses dependency manifests to flag outdated or unsupported

cryptographic libraries. The results are stored in a graph database so that relationships between all discovered assets can be queried and reasoned about.

The risk-graph construction layer maps six categories of nodes — code files, cryptographic usage sites, certificates, network endpoints, libraries, and sensitive data — connected by seven relationship types (for example, “this code uses this certificate” or “this certificate protects this data”). Each node receives a starting heuristic risk score from a published, checkable formula that combines measurable factors: data sensitivity, asset criticality, internet exposure, cryptographic algorithm strength, known CVEs, dependency centrality, and migration effort. This heuristic is deliberately kept free of any assumption about when large-scale quantum computers will arrive, so that it remains auditable and reproducible.

5.1 Weak Supervision, the Role of the GNN, and the Leakage Ablation

A central design decision in AgileGraph is that the heuristic risk score is treated as an initial, weak-supervision estimate rather than as ground truth. Using that same score both as a node feature and as the training label creates a real risk that the model reproduces the label directly rather than learning from graph structure. To test this directly, the model is trained in two configurations: (a) with the heuristic score included as a node feature, and (b) with it withheld and only structural/categorical node attributes retained. If the withheld-feature model still recovers most of the ranking improvement over the heuristic baseline, that is direct evidence the gain comes from graph propagation; if performance collapses without the heuristic feature, that is reported honestly as a limitation. This ablation is reported as a first-class table in the evaluation section.

The GNN's contribution is evaluated, not assumed. After training, the project compares the heuristic-only ranking, the GNN-refined ranking, and an existing open tool's flat ranking (CBOMkit) to quantify how much graph propagation actually changes prioritization, reports bootstrap confidence intervals on the agreement metrics, and runs a paired significance test (e.g., a permutation test on rank-agreement deltas) to establish that observed differences are unlikely to be noise at the sample sizes involved. Specific case studies are reported where the GNN reorders assets relative to the heuristic because of relational structure the heuristic cannot see in isolation.

5.2 Expert Validation

A held-out sample of 150–200 assets is independently annotated by a panel of at least four security professionals (drawn from faculty, industry contacts, or senior security-club members with relevant experience) using a categorical scale (High / Medium / Low risk), collected without reference to the heuristic formula's internal weights. Before comparing the model to the panel, inter-annotator agreement is computed among the experts themselves (weighted Cohen's kappa for pairs, Fleiss' kappa across the full panel); this establishes a reliability ceiling so that a strong GNN-vs-expert agreement score can be interpreted against how much experts agree with each other, rather than in isolation. Final labels are taken as majority vote (or adjudicated on disagreement) and used solely to evaluate whether the GNN-refined ranking agrees better with expert judgment than the flat heuristic and CBOMkit baselines, using weighted Cohen's kappa and macro F1, with bootstrap confidence intervals. The expert-labeled sample is never used to construct or tune the heuristic formula, preserving a clean separation between the weak training signal and the evaluation ground truth.

5.3 Mosca Readiness Index

Because the projected timeline for large-scale quantum computers cannot be scientifically established, it is kept out of the operational risk score entirely. Instead, AgileGraph separately reports a Mosca Readiness Index, based on Mosca's inequality ($x + y > z$, where x is the required data confidentiality lifetime, y is estimated migration duration, and z is an assumed quantum-capability horizon). This index is presented as an organization-configurable strategic planning indicator on the dashboard, with z exposed as an adjustable parameter rather than a fixed constant, so that the model's core methodology does not depend on an unprovable assumption.

5.4 Weight Justification

The weights used in the heuristic formula are drawn from an initial literature-informed starting point and then stress-tested two ways: (a) a sensitivity analysis in which each weight is perturbed by approximately ±10–20% and the resulting migration ranking is checked for stability, and (b) a lightweight pairwise-comparison survey (an “AHP-lite” exercise) run with the same expert panel recruited for Section 5.2. Where the survey-derived weights diverge materially from the literature-informed starting weights, the divergence and its effect on the final ranking are reported explicitly. This is a scoped, defensible justification appropriate to a B.Tech project timeline, and is named as a limitation in the final writeup.

5.5 Explainability Mechanism

Plain-language explanations are generated from two combined sources: (1) GATv2 attention weights (or GNNExplainer subgraph attribution where attention weights alone are insufficiently sparse to interpret) identify which neighboring nodes and relationship types most influenced a given risk score, and (2) a decomposition of the underlying heuristic factors (data sensitivity, exposure, CVE presence, algorithm strength, centrality, effort) shows which measurable inputs dominate the base score. The dashboard presents both together — why the graph model raised an item's priority, and why the base heuristic scored it that way — so that a security team is not given a single opaque number without a stated mechanism.

5.6 Scanning Scope and Authorization

Live TLS/certificate scanning is restricted to infrastructure the team owns or has explicit written authorization to test, and/or passive analysis via public Certificate Transparency logs, which does not require active probing of third-party systems. This scope is stated explicitly in the methodology, both because unauthorized active scanning of systems the team does not control raises legal and ethical concerns, and because reviewers at security venues routinely expect an explicit authorization statement covering any active-scanning component.

5.7 Presentation Layer

The presentation layer is a FastAPI backend and a React-based web dashboard where a user scans a project and receives a ranked migration to-do list, each item annotated with a plain-English explanation and an estimate of how much overall risk is reduced by addressing it, alongside the separate Mosca Readiness Index view.

6. System Requirements

6.1 Hardware Requirements

- Development workstation/laptop with a minimum of 16 GB RAM and a multi-core processor.
- GPU (NVIDIA, CUDA-compatible) recommended for accelerated GNN training, though not strictly required for the target dataset scale.

- Stable internet connectivity for certificate/TLS scanning of authorized live endpoints and passive CT-log queries.

6.2 Software Requirements

- Operating System: Linux / Windows with WSL2 / macOS
- Programming Languages: Python 3.10+, JavaScript/TypeScript
- AI/ML Libraries: PyTorch, PyTorch Geometric (GATv2Conv)
- Explainability: GNNExplainer (PyG), attention-weight extraction utilities
- Code Scanning: Semgrep and custom analysis scripts
- Certificate/Network Scanning: tlsx, testssl.sh, sslyze, Certificate Transparency (CT) logs
- Baseline Comparison Tooling: CBOMkit (open-source CBOM generation, used as an external baseline ranking)
- Graph Storage: Neo4j or NetworkX
- Backend Framework: FastAPI
- Frontend Framework: React with Tailwind CSS
- Evaluation Tools: scikit-learn (weighted Cohen's kappa, macro F1), statsmodels/SciPy (bootstrap confidence intervals, permutation significance testing), matplotlib
- Version Control: Git / GitHub

7. Expected Outcomes

- A working end-to-end prototype that scans a codebase and live infrastructure and produces a ranked, explainable PQC migration list.
- An open, documented heuristic risk-scoring formula, justified by both sensitivity analysis and an AHP-lite expert survey, and a heterogeneous risk graph populated from 8–10 real open-source projects across at least three languages and two size tiers.
- A trained GATv2-based risk-propagation model, explicitly framed as refining a weak-supervision heuristic, with a leakage ablation (with/without heuristic as input feature) and a

quantified, statistically tested comparison of heuristic-only, rule-based, CBOMkit-derived, and GNN-refined rankings.

- An expert-reviewed validation of a held-out sample of 150–200 items using categorical (High/Medium/Low) annotations from a panel of at least four experts, reported with inter-annotator reliability, weighted Cohen's kappa, macro F1, and bootstrap confidence intervals.
- An explainability layer combining attention/GNNExplainer attribution with heuristic factor decomposition, and a separate Mosca Readiness Index module kept outside the GNN training loop.
- A functional web dashboard demonstrating the complete workflow from scan to prioritized migration output.
- A draft research paper documenting the graph model, the weak-supervision risk formula, and the evaluation results.

8. Applications

- Enterprise post-quantum cryptography migration planning across banking, healthcare, telecom, and government sectors.
- Security operations center (SOC) and governance-risk-compliance (GRC) tooling for continuous cryptographic risk monitoring.
- Cybersecurity consulting engagements that require a defensible, explainable migration roadmap for clients.
- Academic and research use as an open, checkable alternative to closed vendor risk-scoring formulas.
- Supply-chain security, complementing existing Software Bill of Materials (SBOM) practices with a Cryptography Bill of Materials (CBOM)-aligned view [9].

9. Future Scope

- Extending code-scanning coverage to additional languages (C/C++, Rust, JavaScript) and cloud-native/IaC configurations.

- Integrating continuous, real-time monitoring with SIEM/SOAR platforms rather than point-in-time scans.
- Incorporating newly standardized PQC algorithms (such as HQC) and updated NIST guidance as they are released.
- Simulating and comparing multiple migration-order strategies at organizational scale, including budget and staffing constraints.
- Extending expert validation from categorical to ordinal ranking annotations if a larger expert panel becomes available, enabling rank-correlation metrics (Spearman, NDCG) alongside the categorical agreement metrics.
- Piloting the system with an industry partner to validate the risk-scoring formula against real production incident data.

10. Project Timeline

Month	Phase	Deliverable
Month 1	Planning + Design	Graph layout finalized; heuristic risk-scoring formula and weak-supervision methodology agreed with advisor; annotation protocol defined; expert panel of 4-6 security professionals recruited and a pairwise-comparison (AHP-lite) survey run to justify heuristic weights
Month 2	Building the Scanners	Code, certificate, and dependency scanners built; graph populated from 8-10 real projects across 3+ languages and 2+ size tiers; heuristic scores computed; live scanning restricted to owned/authorized test infrastructure, documented in an ethics/authorization note
Month 3	Training the AI Model	GATv2 model trained using heuristic as weak supervision/initialization; heuristic-only and rule-based baselines built; leakage ablation run (GNN with vs. without base_risk as input feature)
Month 4	Testing + Validation	Expert-reviewed categorical (High/Medium/Low) sample of 150-200 assets collected independently across all annotators; inter-annotator agreement (kappa) reported as a reliability ceiling; GNN-refined ranking compared against heuristic baseline and against a CBOMkit-derived flat ranking using kappa/F1, with bootstrap confidence intervals and paired significance testing
Month 5	Dashboard + Explainability	Web dashboard built showing the ranked migration list with GNNExplainer/attention-weight-derived explanations, heuristic factor decomposition, and a separate Mosca Readiness Index view

Month 6	Writeup	Final results, ablation tables, case studies of GNN reorderings, limitations section, and paper draft completed

11. References

[1] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Federal Information Processing Standards Publication FIPS 203, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024.

[2] NIST, “Module-Lattice-Based Digital Signature Standard,” Federal Information Processing Standards Publication FIPS 204, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024.

[3] NIST, “Stateless Hash-Based Digital Signature Standard,” Federal Information Processing Standards Publication FIPS 205, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024.

[4] K. F. Hasan, L. Simpson, M. A. R. Baee, C. Islam, Z. Rahman, W. Armstrong, P. Gauravaram, and M. McKague, “A framework for migrating to post-quantum cryptography: Security dependency analysis and case studies,” IEEE Access, vol. 12, pp. 23427–23450, 2024, doi: 10.1109/ACCESS.2024.3360412.

[5] N. Alnahawi, N. Schmitt, A. Wiesmaier, and A. Heinemann, “On the state of crypto-agility,” Cryptology ePrint Archive, 2023.

[6] C. Näther, D. Herzinger, S.-L. Gazdag, J.-P. Steghöfer, S. Daum, and D. Loebenberger, “Migrating software systems towards post-quantum cryptography — a systematic literature review,” arXiv preprint arXiv:2404.12854, 2024.

[7] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?,” in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[8] S. Motie and B. Raahemi, “Financial fraud detection using graph neural networks: A systematic review,” Applied Soft Computing, vol. 149, p. 110984, 2023.

[9] OWASP Foundation, “Authoritative Guide to Cryptography Bill of Materials (CBOM),” CycloneDX Project, 2024.

[10] A. Shaw, “Quantum-Safe Code Auditing: LLM-Assisted Static Analysis and Quantum-Aware Risk Scoring for Post-Quantum Cryptography Migration,” arXiv preprint arXiv:2604.00560, 2026.